Telekommunikációs Hálózatok

Számítógépes Hálózatok

3. gyakorlat

Protokoll

BEADANDÓ II.

Beadandó - Protokoll

A feladat személyenként randomizált.

Elérési link: https://ggombos.web.elte.hu/halobeadando/2-protokoll/

(a link megtalálható a tms rendszer feladatánál is)

Példa feladat:

Neptun azonosító: AFBDPK Add a házit!

Feladat1

A paraméterben kapott bináris fájlokat olvasssuk be és irassuk ki az első soruk tartalmát a standard outputra!
- Parameter1 formatuma: 9 hosszú string, integer, float
- Parameter2 formatuma: float, bool, karakter
- Parameter3 formatuma: karakter, integer, 9 hosszú string
- Parameter4 formatuma: float, 9 hosszú string, bool

Feladat2

Írd ki a stdout-ra a következő értéket bináris formátumban (a pack eredményét)! A string hosszát a szöveg mögött lévő szám jelzi!
Használandó struct paraméterek: f, i, c, ?, Xs (ahol a X a string hossza, pl: 3s)
- "elso"(12), 70, True
- 73.5, False, 'X'
- 61, "masodik"(10), 80.9
- 'Z', 92, "harmadik"(13)

Leadás: A program leadása a TMS rendszeren keresztül **.zip** formátumban, amiben egy **client.py** szerepeljen!

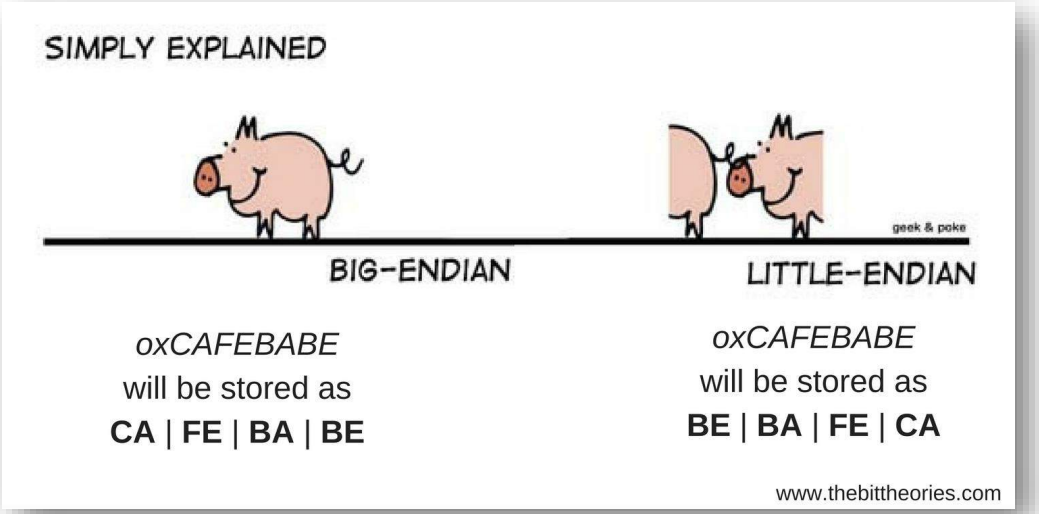
portok, kódolás

KOMMUNIKÁCIÓS CSATORNA I.

Bájtsorrendek

00000000 00000000 00000100 00000001

Address	Big-Endian representation of 1025	Little-Endian representation of 1025
00	00000000	00000001
01	00000000	00000100
02	00000100	00000000
03	00000001	00000000



- Nézzük meg, hogy a saját gépünk milyen kódolást használ! sys.byteorder
- A hálózati bájt sorrend big-endian (a magasabb helyi értéket tartalmazó bájt van elől)
- 16 és 32 bites pozitív számok kódolása
  - htons(), htonl() – host to network short / long
  - ntohs(), ntohl() – network to host short / long

Port	Service name	Transport protocol
20, 21	File Transfer Protocol (FTP)	TCP
22	Secure Shell (SSH)	TCP and UDP
23	Telnet	TCP
25	Simple Mail Transfer Protocol (SMTP)	TCP
50, 51	IPSec	
53	Domain Name System (DNS)	TCP and UDP
67, 68	Dynamic Host Configuration Protocol (DHCP)	UDP
69	Trivial File Transfer Protocol (TFTP)	UDP
80	HyperText Transfer Protocol (HTTP)	TCP
110	Post Office Protocol (POP3)	TCP
119	Network News Transport Protocol (NNTP)	TCP
123	Network Time Protocol (NTP)	UDP
135-139	NetBIOS	TCP and UDP
143	Internet Message Access Protocol (IMAP4)	TCP and UDP
161, 162	Simple Network Management Protocol (SNMP)	TCP and UDP
389	Lightweight Directory Access Protocol	TCP and UDP
443	HTTP with Secure Sockets Layer (SSL)	TCP and UDP
989, 990	FTP over SSL/TLS (implicit mode)	TCP
3389	Remote Desktop Protocol	TCP and UDP

Pár neves port

A port jelenti a kommunikáció végpontját, azaz bemeneti-kimeneti "kapuk", melyeken folyik az adatátvitel.

Port számok

- Bizonyos protokollokhoz tartoznak fix portszámok, konstansok (szállítási protokollok)!
- getservbyport()

```
socket.getservbyport(22)
```

Feladat: port szolgáltatások

- Írassuk ki a 1..100-ig a portokat és a hozzájuk tartozó protokollokat!

- Ha nem tartozik hozzá, írajuk ki saját üzenetet, küldönben pedig a port nevét.

python socket, TCP

KOMMUNIKÁCIÓS CSATORNA II.

Pyton socket, host név feloldás

- Socket csomag használata

```
import socket
```

- gethostname()

```
hostname = socket.gethostname()          #megadja a PC nevét
```

- gethostbyname()

```
hostname = socket.gethostbyname(‚www.example.org’)    #IP címet
```

- gethostbyname_ex()

```
hostname, aliases, addresses = socket.gethostbyname_ex(host)
```

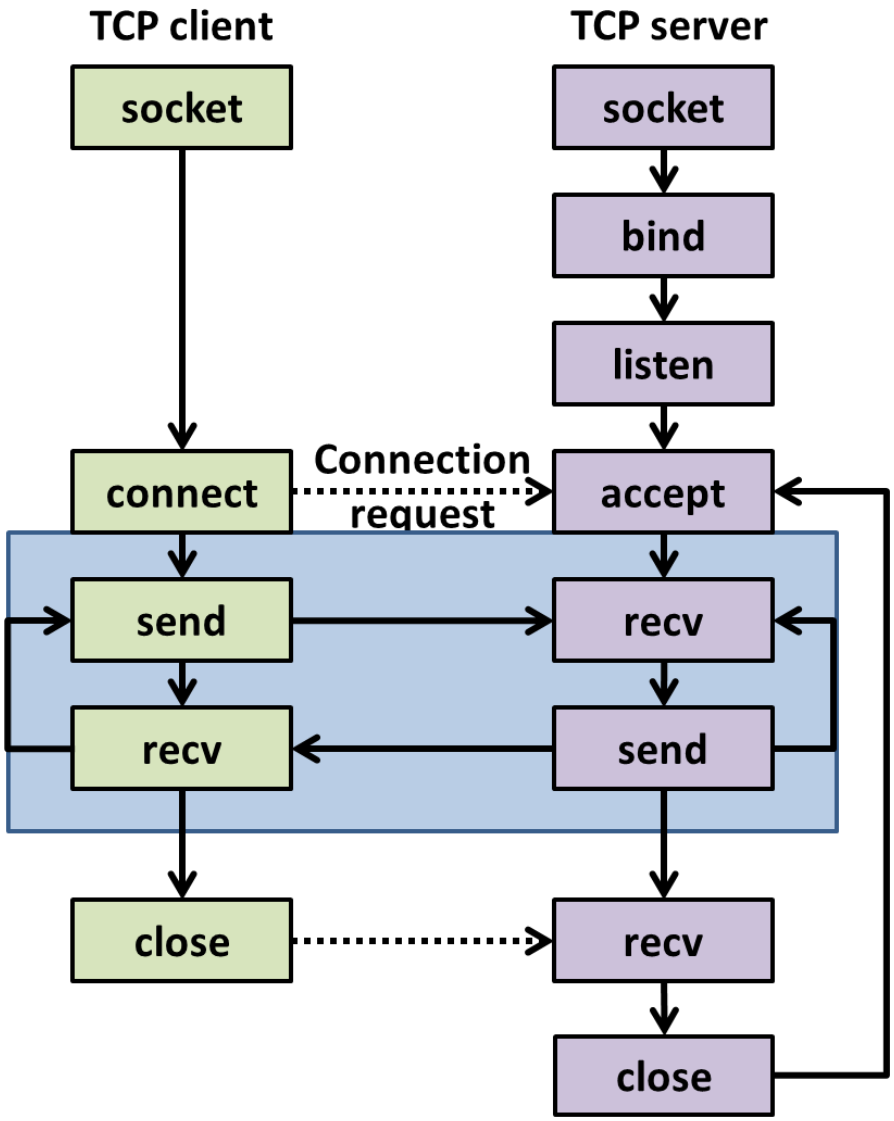
- gethostbyaddr()

```
hostname, aliases, addrs = socket.gethostbyaddr('157.181.161.79')
```

A kommunikációs csatorna kétféle típusa

- **Kapcsolat-orientált modell (analógia: telefonbeszélgetés)**
  - csomagok megérkeznek jó sorrendben
  - ilyen protokoll a TCP
  - kapcsolódó típus: stream socket
- Kapcsolat-nélküli modell (analógia: postai levelezés)
  - csomagok nem biztos, hogy sorrend helyesen érkeznek, sőt el is veszhetnek
  - előnye a jobb teljesítmény
  - ilyen protokoll a UDP
  - kapcsolódó típus: datagram socket

TCP

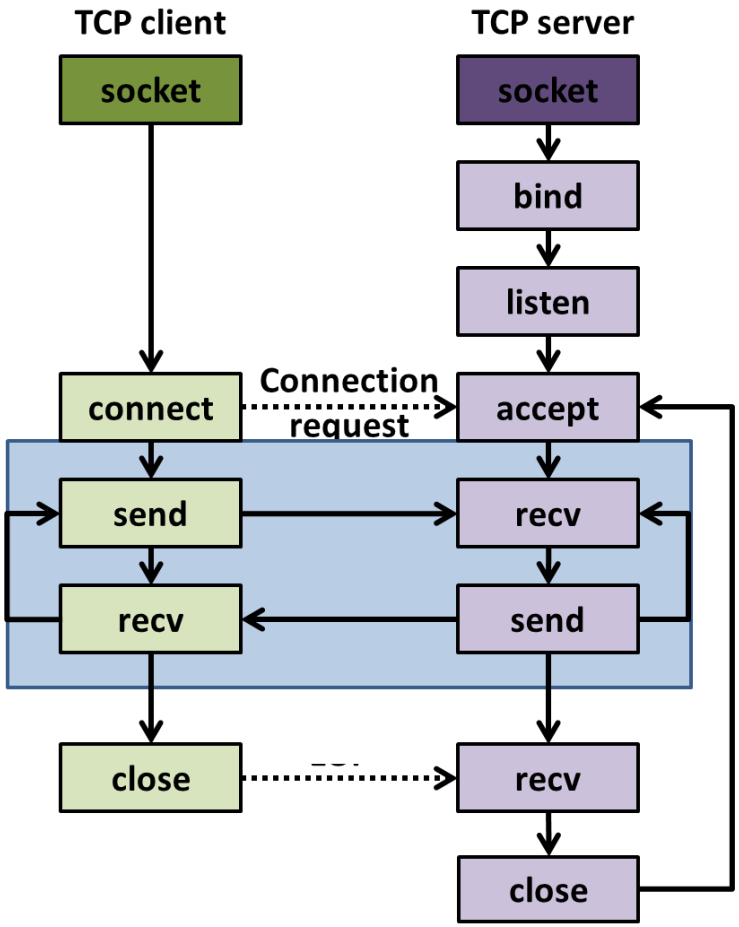

TCP client
TCP server
socket
socket
bind
listen
connect
Connection request
accept
send
recv
recv
send
close
recv
close

Socket leíró beállítása

- socket.**socket**([*family*
  [, *type*
  [, *proto*]]])
- *family* : socket.AF_INET → IPv4 (AF_INET6 → IPv6)
- *type* : socket.SOCK_STREAM → TCP
- *proto* : 0

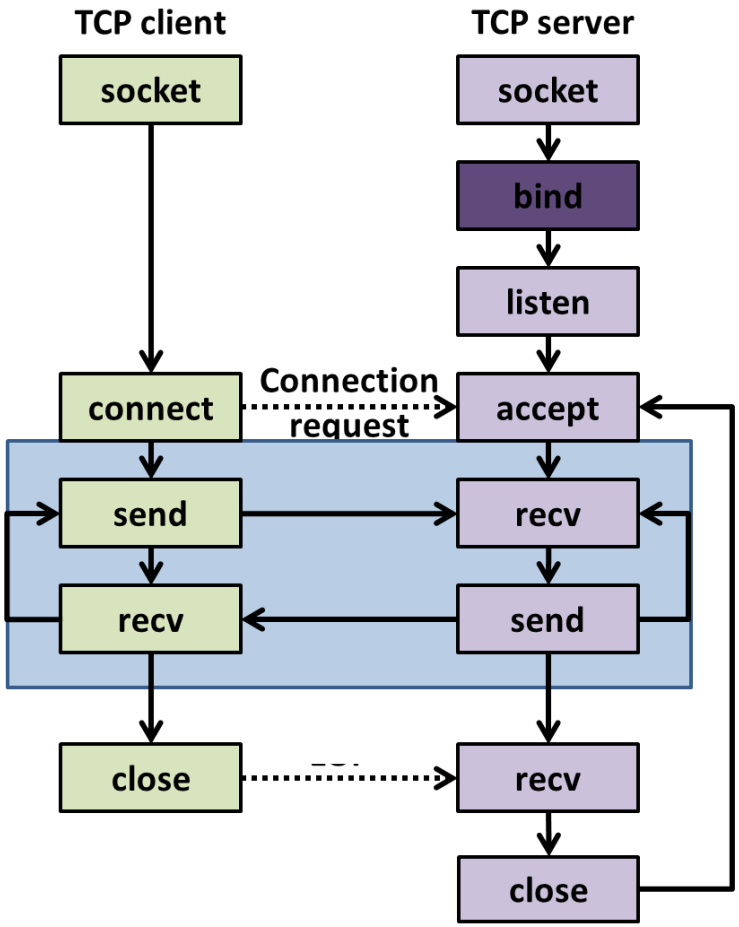
(alapértelmezett protokoll lesz)

- visszatérési érték: egy socket objektum, amelynek a metódusai a különböző socket rendszer hívásokat implementálják



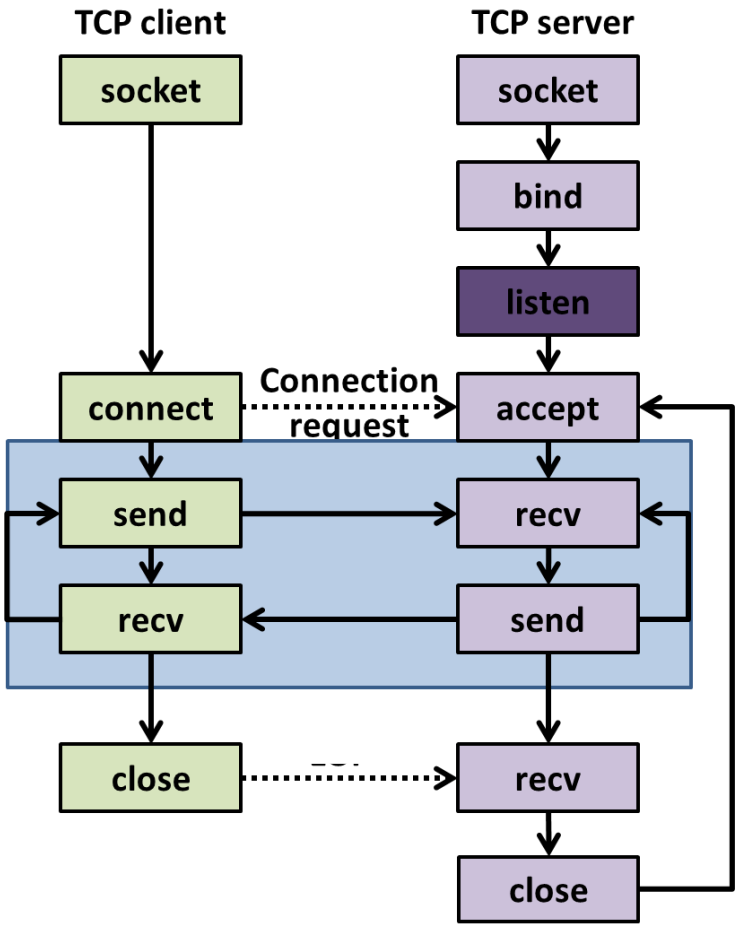
Bindolás

- socket.socket.**bind**(*address*)
- A socket objektum metódusa
- *address* : egy tuple, amelynek az első eleme egy hosztnév vagy IP cím (sztring reprezentációval), második eleme a portszám



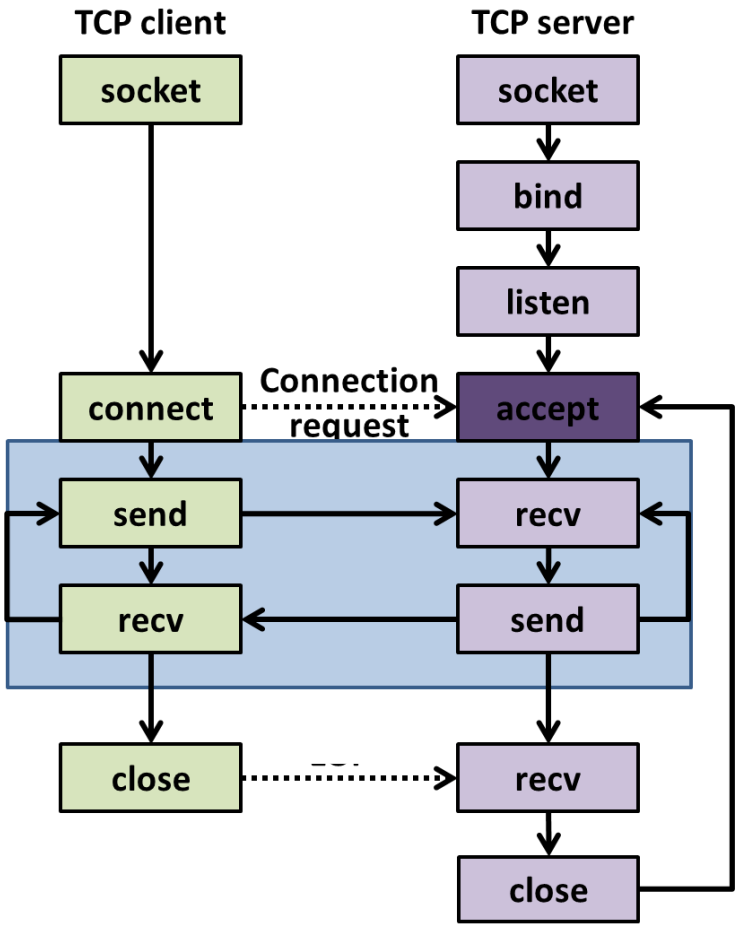
Listen

- socket.socket.**listen**(*backlog*)
- A socket objektum metódusa
- *backlog* : egy egész szám, ennyi kapcsolódási igény várakozhat a sorban



Accept

- socket.socket.**accept**()
- A socket objektum metódusa
- A szerver elfogadhatja a kezdeményezett kapcsolatokat
- visszatérési érték: egy tuple,
  - amelynek az első eleme egy **új socket objektum** a kapcsolaton keresztüli adatküldésre és fogadásra
  - második eleme a kapcsolat túlsó végén lévő cím



TCP

- socket()

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

- bind()

```
server_address = ('localhost', 10000)
sock.bind(server_address)
```

- listen()

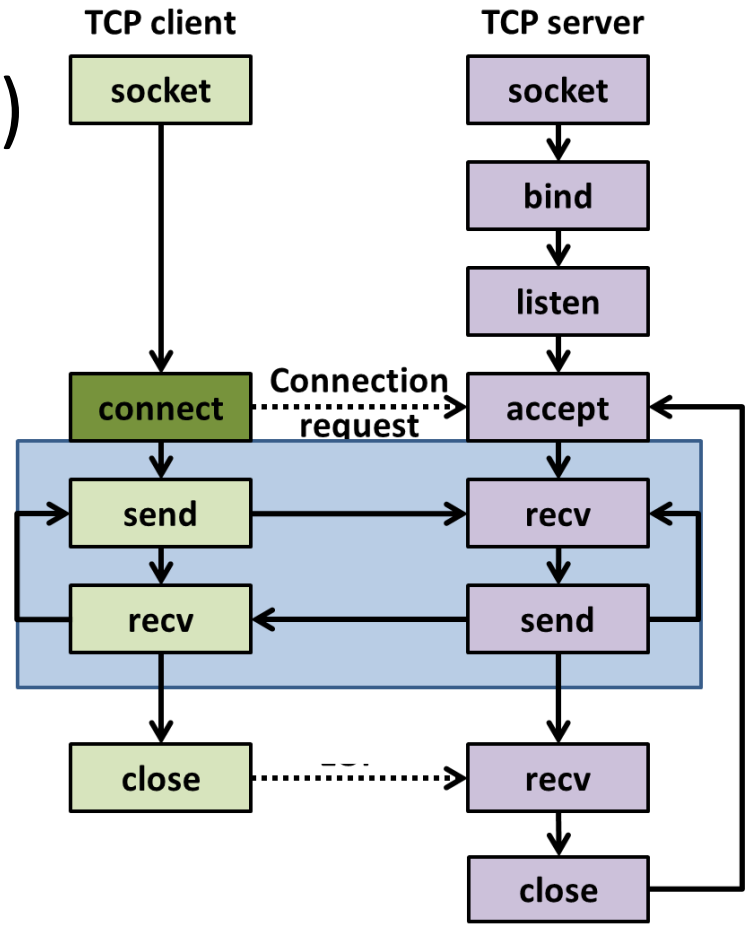
```
sock.listen(1)
```

- accept()

```
connection, client_address = sock.accept()
```

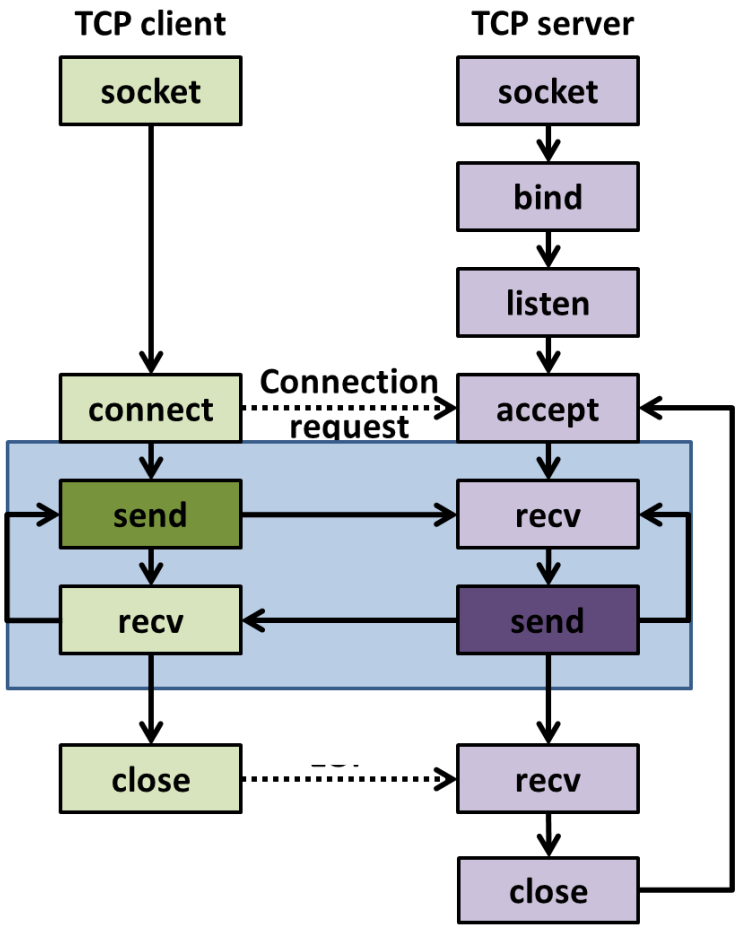
Connect

- socket.socket.**connect**(*address*)
- A socket objektum metódusa
- Kapcsolódás megkezdése egy távoli sockethez az *address* címen (ezt például kezdeményezheti egy kliens)
- Az *address* típusát ld. a **bind** függvénynél



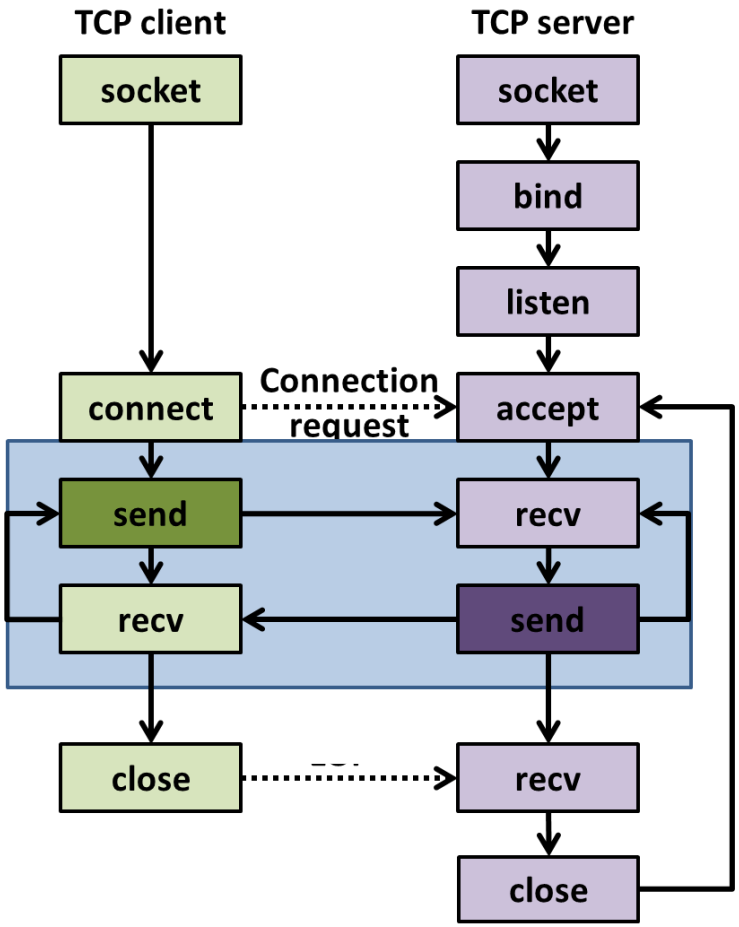
Send

- socket.socket.**send**(*string* *[, flags]*)
- A socket objektum metódusa
- Adatküldés (*string*) a socketnek
- *flags* : 0 (nincs flag meghatározva)
- A socketnek előtte már csatlakozni kellett a távoli sockethez!
- visszatérési érték: az átküldött bájtok száma
  - az alkalmazásnak kell ellenőrizni, hogy minden adat átment-e
  - ha csak egy része ment át: újra kell küldeni a maradékot



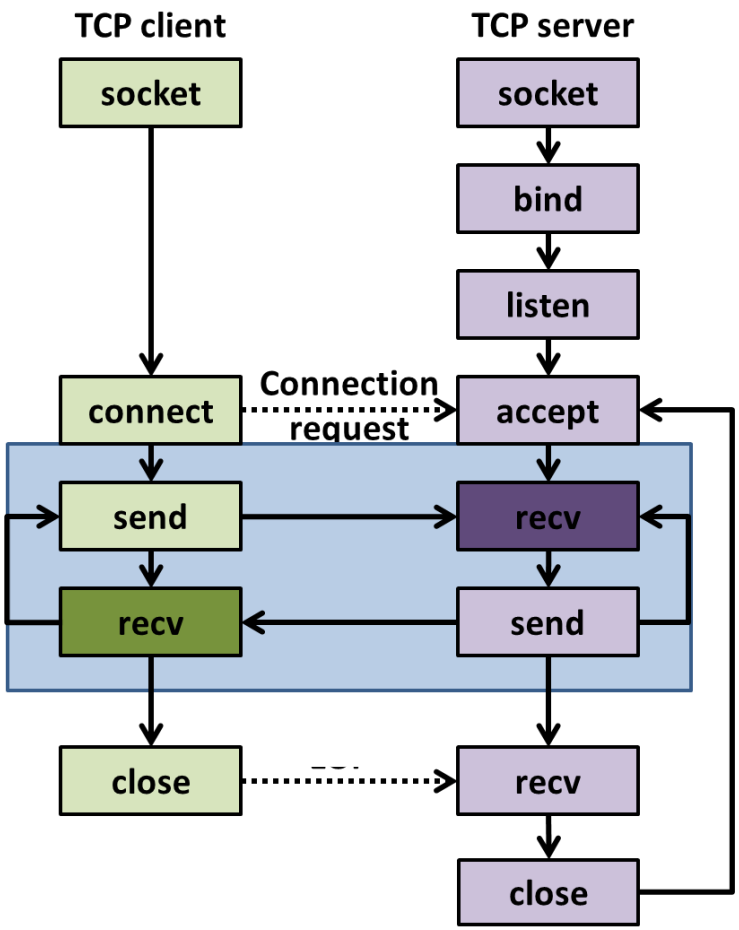
Sendall

- socket.socket.**sendall**(
  string
  [, flags])
- A socket objektum metódusa
- Az előzőhöz hasonló
- A különbség: addig küldi az adatot a *string*-ből, ameddig az összes át nem ment, vagy hiba nem történt (ebben az esetben már nem lehet kideríteni, hogy mennyi adat ment át)
- visszatérési érték: None, ha sikeres volt



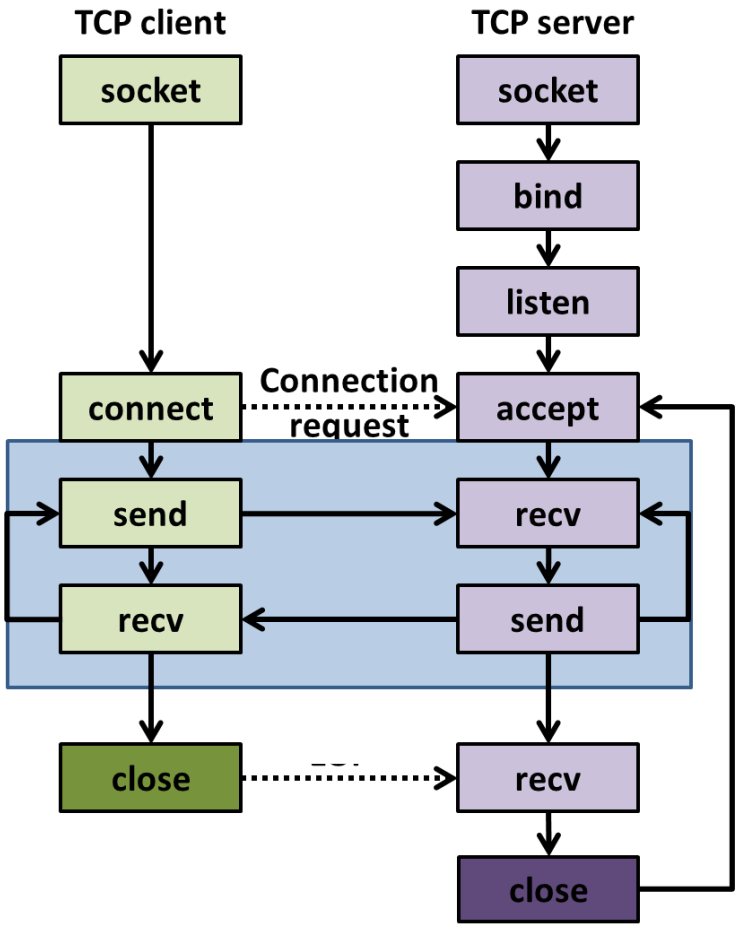
Recv

- socket.socket.**recv**(*bufsize* *[, flags]*)
- A socket objektum metódusa
- Üzenet fogadása
- *bufsize* : a max. adatmennyiség, amelyet egyszerre fogadni fog
- *flags* : 0 (nincs flag meghatározva)
- visszatérési érték: a fogadott adat sztring reprezentációja



Close

- socket.socket.**close**()
- A socket objektum metódusa
- A socket lezárása:
  - az összes további művelet a socket objektumon el fog bukni
  - a túlsó végpont nem fog több adatot kapni
  - ez el fogja engedni a kapcsolathoz tartozó erőforrásokat, **de** nem feltétlen zárja le azonnal (ha erre szükség van, akkor érdemes **shutdown** hívást a **close** elé tenni)



TCP

- send(), sendall()

```
connection.sendall(data)                    #python 2.x
```

```
connection.sendall(data.encode())           #python 3.x
```

- recv()

```
data = connection.recv(16)                  #python 2.x
```

```
data = connection.recv(16).decode()         #python 3.x
```

- close()

```
connection.close()
```

- connect()

```
server_address = (,localhost', 10000)
sock.connect(server_address)
```

Feladat: Hello

- Készítsünk egy egyszerű kliens-server alkalmazást, ahol a kliens elküld egy ‚Hello server' üzenetet, és a szerver pedig válaszol neki egy ‚Hello kliens' üzenettel!

- Változtassuk meg hogy ne az előre megadott portot adjuk, hanem egy tetszőlegeset kapjunk az oprendszertől! (sys.argv[1])

Feladat: Számológép

Készítsünk egy szerver-kliens alkalmazást, ahol a kliens elküld 2 számot és egy operátort a szervernek, amely kiszámolja és visszaküldi az eredményt. A kliens üzenete legyen struktúra.

VÉGE
KÖSZÖNÖM A FIGYELMET!